

Relazione tecnica — OpenTravel: pianificatore di percorsi ottimali

1. Introduzione

OpenTravel è un'applicazione web client-side (nessun backend proprio) che permette all'utente di:

1. inserire una serie di tappe (cliccando sulla mappa o cercando un indirizzo),
2. calcolare l'**ordine ottimale** in cui visitarle (un problema del commesso viaggiatore, *Traveling Salesman Problem* — TSP, aperto, senza vincolo di ritorno al punto di partenza),
3. visualizzare il percorso su mappa, la distanza/durata totale e il **profilo altimetrico** lungo il tragitto.

L'intera logica gira nel browser dell'utente: non c'è un server applicativo, un database o un backend REST proprietario. Tutti i dati (geocoding, matrice dei tempi, geometria del percorso, altimetria) vengono ottenuti da **API pubbliche di terze parti**, chiamate direttamente dal client via **fetch**.

2. Struttura del progetto

L'app è composta da tre soli file, secondo il classico pattern statico HTML/CSS/JS:

File	Ruolo
<code>index.html</code>	Struttura del DOM: header, sidebar dei controlli, contenitore mappa, profilo altimetrico, overlay di caricamento
<code>style.css</code>	Tema visivo (light/dark), layout responsive, variabili CSS usate anche da JS (es. colori del grafico altimetrico)
<code>app.js</code>	Tutta la logica applicativa, incapsulata in un'unica classe <code>RoutePlanner</code>

Non è presente alcun bundler, framework SPA o build step: gli script sono inclusi come `<script>` classici e la libreria di mappe **Leaflet** è caricata da CDN (unpkg.com), insieme ai font Google (Space Grotesk, Public Sans, IBM Plex Mono).

```
<link rel="stylesheet" href="https://unpkg.com/leaflet/dist/leaflet.css" />
...
<script src="https://unpkg.com/leaflet/dist/leaflet.js"></script>
<script src="app.js"></script>
```

3. Architettura logica: la classe `RoutePlanner`

All'avvio (`const app = new RoutePlanner()`), viene istanziata un'unica classe che accentra:

- **lo stato applicativo:** `points` (le tappe), `markers` (i marker Leaflet associati), `matrix` (matrice tempi/distanze), `cache` (cache dei risultati OSRM Table), `routeLayer` (layer geoJSON del percorso disegnato), `elevationData`, `isDark` (tema).
- **l'inizializzazione della mappa** Leaflet centrata sulle Marche (`[43.7, 12.6]`, zoom 6), con tile layer standard OpenStreetMap.
- **gli event listener:** click sulla mappa per aggiungere una tappa, drag dei marker, ridimensionamento del pannello altimetrico via trascinamento, `ResizeObserver` per adattare mappa e grafico quando cambia la dimensione del contenitore.

L'interfaccia (`index.html`) invoca i metodi della classe tramite attributi `onclick` (es. `onclick="app.addByAddress()"`), quindi non c'è un vero binding reattivo: ogni azione richiama un metodo che aggiorna manualmente lo stato e poi il DOM (pattern "manual DOM manipulation", non un framework reattivo tipo React/Vue).

3.1 Flusso di interazione principale

1. L'utente aggiunge tappe cliccando sulla mappa (`addPoint`) o cercando un indirizzo (`addByAddress` → `geocode`).
2. Ogni tappa è un oggetto `{ id, lat, lng }`; ad ogni modifica (aggiunta, rimozione, drag) l'app invalida il percorso già calcolato (`clearRoute`) e nasconde il profilo altimetrico.
3. Premendo **"Calcola"** (`calculate`):
 - viene costruita/recuperata dalla cache la **matrice tempi/distanze** tra tutte le tappe (`buildMatrix`, via OSRM Table API);
 - in base al numero di tappe, viene scelto l'algoritmo di ottimizzazione (esatto o euristico, vedi §5);
 - il percorso ottimale (sequenza di indici) viene tracciato realmente su strada (`draw`, via OSRM Route API) e disegnato sulla mappa;
 - viene calcolato il profilo altimetrico (`fetchElevationProfile`, via Open-Elevation API) e disegnato su `<canvas>`.

4. Le API esterne utilizzate

Tutte le chiamate sono `fetch` dirette dal browser verso servizi pubblici gratuiti dell'ecosistema OpenStreetMap. Di seguito modalità d'uso e struttura delle risposte JSON effettivamente consumate dal codice.

4.1 Nominatim — Geocoding (indirizzo → coordinate)

Endpoint: <https://nominatim.openstreetmap.org/search>

Uso nel codice (`geocode`):

```
GET /search?format=json&q=<indirizzo urlencoded>&limit=1
Header: User-Agent: TSP-App
```

Il metodo implementa un **semplice rate-limiting client-side**: prima di ogni chiamata verifica che sia trascorso almeno 1,1 secondi dall'ultima richiesta (`this.lastNominatim`), attendendo con `setTimeout` se necessario — rispettando così la policy di utilizzo di Nominatim, che impone un massimo di 1 richiesta al secondo per IP.

Struttura della risposta JSON (array di risultati; l'app usa solo il primo, `limit=1`):

```
[
  {
    "place_id": 123456,
    "licence": "Data © OpenStreetMap contributors, ODbL 1.0",
    "osm_type": "way",
    "osm_id": 987654,
    "lat": "43.7263",
    "lon": "12.4365",
    "display_name": "Urbino, Pesaro e Urbino, Marche, Italia",
    "boundingbox": ["43.72", "43.73", "12.43", "12.44"]
  }
]
```

Dal risultato l'app estrae solo `lat` e `lon` (convertiti in numero con l'operatore unario `+`):

```
return { lat: +d[0].lat, lng: +d[0].lon };
```

Se l'array è vuoto, l'indirizzo non è stato trovato e viene mostrato un `alert`.

4.2 OSRM Table Service — matrice tempi/distanze

Endpoint: <https://router.project-osrm.org/table/v1/driving/{coordinate}>

Uso nel codice (`buildMatrix`):

```
GET /table/v1/driving/lng1,lat1;lng2,lat2;...;lngN,latN?
annotations=duration,distance
```

Nota: OSRM vuole le coordinate in formato `longitude, latitude` (invertito rispetto alla convenzione lat/lng usuale), e le tappe sono separate da `;`.

Questa è la chiamata più "costosa" concettualmente perché restituisce una matrice **N×N** (tutte le coppie origine-destinazione), necessaria per calcolare il TSP senza dover fare N^2 chiamate separate. Per questo il risultato viene **cachato** in `this.cache` usando come chiave la serializzazione JSON delle coordinate dei punti: se l'insieme di tappe non cambia, la matrice non viene richiesta di nuovo.

Struttura della risposta JSON:

```
{
  "code": "Ok",
  "durations": [
    [0, 620.4, 1830.7],
    [615.2, 0, 1290.1],
    [1810.5, 1275.9, 0]
  ],
  "distances": [
    [0, 8500.2, 25400.8],
    [8490.1, 0, 18200.3],
    [25100.6, 18150.7, 0]
  ],
  "sources": [ { "location": [12.43, 43.73], "name": "" }, ... ],
  "destinations": [ { "location": [12.43, 43.73], "name": "" }, ... ]
}
```

L'app usa solo `durations` (in secondi) e `distances` (in metri), salvate come:

```
this.matrix = { durations: d.durations, distances: d.distances };
```

`durations[i][j]` rappresenta il tempo di viaggio stimato in auto dalla tappa `i` alla tappa `j`; è la matrice usata come "costo" da tutti gli algoritmi di ottimizzazione.

4.3 OSRM Route Service — geometria reale del percorso

Endpoint: <https://router.project-osrm.org/route/v1/driving/{coordinate}>

Uso nel codice (metodi `draw` e `fetchElevationProfile`), una volta che l'ordine ottimale delle tappe è stato deciso:

```
GET /route/v1/driving/lng1,lat1;...;lngN,latN?
overview=full&geometries=geojson&steps=true
```

- `overview=full` chiede la geometria completa (non semplificata) del tracciato stradale reale;
- `geometries=geojson` restituisce la polilinea come coordinate GeoJSON invece che come stringa "polyline" codificata;
- `steps=true` richiede anche le singole istruzioni di navigazione (turn-by-turn), anche se l'app attuale non le mostra a video.

Struttura della risposta JSON (semplificata):

```
{
  "code": "Ok",
  "routes": [
    {
      "distance": 45230.5,
      "duration": 3120.8,
      "geometry": {
        "type": "LineString",
        "coordinates": [
          [12.4365, 43.7263],
          [12.4390, 43.7280],
          "... centinaia/migliaia di punti ..."
        ]
      },
      "legs": [
        {
          "distance": 20500.1,
          "duration": 1500.2,
          "steps": [ /* istruzioni dettagliate per tappa */ ]
        }
      ]
    }
  ],
  "waypoints": [ { "location": [12.43, 43.73], "name": "Via Roma" }, ... ]
}
```

Dal primo elemento di `routes` l'app usa:

- `geometry` → per disegnare la linea del percorso sulla mappa (L. `geoJSON`);

- **distance** (metri) e **duration** (secondi) → convertiti rispettivamente in km ($\div 1000$) e minuti ($\div 60$) e mostrati nel pannello risultati.

4.4 Open-Elevation — profilo altimetrico

Endpoint: <https://api.open-elevation.com/api/v1/lookup>

Uso nel codice (`fetchElevationProfile`): a differenza delle altre, questa è una **POST** con corpo JSON, perché il numero di punti da interrogare può essere elevato:

```
POST /api/v1/lookup
Content-Type: application/json

{
  "locations": [
    { "latitude": 43.7263, "longitude": 12.4365 },
    { "latitude": 43.7290, "longitude": 12.4410 }
  ]
}
```

Prima di interrogare il servizio, la geometria del percorso (che può contenere migliaia di punti) viene **sotto-campionata a un massimo di ~100 punti**:

```
const step = Math.max(1, Math.floor(points.length / 100));
const sampled = points.filter((_, i) => i % step === 0);
```

questo riduce il carico sull'API pubblica (gratuita e con limiti di rate) mantenendo comunque una risoluzione sufficiente per disegnare un grafico leggibile.

Struttura della risposta JSON:

```
{
  "results": [
    { "latitude": 43.7263, "longitude": 12.4365, "elevation": 451 },
    { "latitude": 43.7290, "longitude": 12.4410, "elevation": 478 }
  ]
}
```

Le quote (**elevation**, in metri) vengono estratte in un array parallelo ai punti campionati. Le distanze cumulative tra i punti sono calcolate **non** dall'API ma localmente, con un'approssimazione euclidea in gradi convertita in km tramite il fattore **111.32** (km per grado di latitudine all'equatore):

```
cumulativeDist += Math.sqrt(dx*dx + dy*dy) * 111.32;
```

Questa è un'approssimazione semplificata (non tiene conto della curvatura terrestre in modo rigoroso, né della diversa lunghezza di un grado di longitudine al variare della latitudine), accettabile per un grafico indicativo ma non per misure di precisione.

Il risultato finale, `{ elevations, distances }`, alimenta la funzione `drawElevationProfile` che disegna il grafico su `<canvas>` (area riempita + linea, con griglia orizzontale, calcolo di dislivello totale, quota minima e massima).

4.5 Riepilogo del flusso di chiamate per un calcolo completo

```
Aggiunta tappe (Nominatim, opzionale, 1 chiamata per indirizzo)
    ↓
buildMatrix()      → OSRM /table  (1 chiamata, cachata)
    ↓
algoritmo TSP (locale, nessuna chiamata di rete)
    ↓
draw(path)         → OSRM /route  (1 chiamata, sul percorso ordinato)
    ↓
fetchElevationProfile(path) → OSRM /route (di nuovo, per riottenere la
                                geometria)
                                → Open-Elevation /lookup (1 chiamata POST)
```

Da notare che `draw()` e `fetchElevationProfile()` richiamano **entrambi** l'endpoint `/route` di OSRM con gli stessi identici parametri, duplicando quindi una chiamata di rete che potrebbe essere riutilizzata (punto di possibile ottimizzazione, vedi §7).

5. Logica degli algoritmi di ricerca del percorso (TSP)

Il problema da risolvere, una volta nota la matrice dei tempi `durations[i][j]`, è trovare l'ordine di visita delle N tappe che **minimizza il tempo totale di viaggio**. Si tratta di un TSP "aperto" (*path*, non *cycle*): non è richiesto tornare al punto di partenza, e non c'è vincolo su quale tappa sia l'ultima.

L'app sceglie strategia in base al numero di tappe, tramite una soglia fissa:

```
this.EXACT_LIMIT = 12;
...
if (n <= this.EXACT_LIMIT) {
    path = this.heldKarp();           // esatto
} else {
    path = this.twoOpt(this.multiStartNN()); // euristico
}
```

(Nota: l'etichetta mostrata all'utente per l'algoritmo esatto è "Concorde (esatto)" — un nome improprio ereditato probabilmente da un altro contesto, dato che l'implementazione reale non usa il risolutore Concorde ma un algoritmo di **programmazione dinamica di Held-Karp**, descritto sotto.)

5.1 Algoritmo esatto: Held-Karp (bitmask DP)

Usato quando il numero di tappe è ≤ 12 . È l'algoritmo classico che risolve il TSP in tempo esponenziale ma molto più rapido della forza bruta ($O(2^n \cdot n^2)$ contro $O(n!)$).

Idea di base: si definisce

```
dp[mask][j] = costo minimo per visitare esattamente l'insieme di tappe
"mask"
               (bitmask a n bit), terminando nella tappa j
```

dove `mask` è un intero le cui bit accese rappresentano le tappe già visitate.

Caso base: partire da ciascuna singola tappa `i` ha costo 0:

```
dp[(1<<i)][i] = 0    // per ogni i
```

Transizione: per ogni sotto-insieme `mask` che include `last`, si prova ad arrivare a `last` da ogni possibile tappa precedente `prev` appartenente a `mask \ {last}`:

```
dp[mask][last] = min su prev ∈ (mask senza last) di:
                    dp[mask senza last][prev] + durations[prev][last]
```

Nel codice questa doppia iterazione è realizzata con tre cicli annidati (`mask`, `last`, `prev`), e ogni stato (`mask`, `last`) è tenuto in una `Map` con chiave stringa "`mask, last`" (anziché un array 2D, probabilmente per semplicità/robustezza in JS con numeri di tappe non fissi). Viene tenuta anche una mappa `parent` per poter **ricostruire il cammino** a ritroso una volta trovato il minimo.

Passo finale: si cerca, tra tutti i possibili "ultimi nodi" `last`, quello che minimizza `dp[fullMask][last]`, dove `fullMask = (1<n)-1` rappresenta "tutte le tappe visitate". Poiché è un TSP aperto, **non si aggiunge il costo di ritorno** all'origine: si prende semplicemente il minimo su tutte le possibili tappe finali.

Ricostruzione del percorso: partendo da (`fullMask`, `bestLast`), si risale tramite la mappa `parent` finché il bitmask non si azzerà, poi si inverte l'array ottenuto (`path.reverse()`), ottenendo la sequenza di indici dall'inizio alla fine.

Complessità: $O(2^n \cdot n^2)$ in tempo e $O(2^n \cdot n)$ in spazio. Con $n = 12$ questo significa circa $2^{12} \times 144 \approx 590.000$ operazioni elementari: del tutto trattabile in JavaScript nel browser in tempo reale. Oltre questa soglia, la crescita esponenziale (raddoppio ad ogni tappa aggiuntiva) renderebbe il calcolo troppo lento — da qui la scelta della soglia `EXACT_LIMIT = 12`.

5.2 Algoritmo euristico: Nearest Neighbor multi-start + 2-opt

Usato quando le tappe sono **più di 12**, per mantenere tempi di calcolo ragionevoli rinunciando alla garanzia di ottimalità assoluta.

5.2.1 Nearest Neighbor (NN)

Per ogni possibile tappa di partenza (`nearestNeighbor(startIndex)`):

1. si parte dalla tappa `startIndex` e la si marca come visitata;
2. ad ogni passo si sceglie, tra le tappe non ancora visitate, quella con il **tempo di percorrenza minimo** dalla tappa corrente (`matrix.durations[last][i]`);
3. si ripete finché tutte le tappe sono state visitate (o finché non se ne trova più una raggiungibile, nel qual caso l'algoritmo si ferma anticipatamente).

È un algoritmo goloso (*greedy*): ottimo localmente ad ogni passo, ma senza garanzia di ottimalità globale, e con complessità $O(n^2)$ per singola esecuzione.

5.2.2 Multi-start

`multiStartNN()` esegue l'algoritmo Nearest Neighbor **partendo da ogni possibile tappa** come punto di origine (`for i in 0..n`), calcola il costo totale di ciascun percorso risultante (`cost(p)`, sommando le durate lungo il cammino), e tiene il migliore. Questo mitiga in parte il rischio che una scelta greedy iniziale porti a un risultato scadente, al costo di $O(n^3)$ complessivo (n esecuzioni di un algoritmo $O(n^2)$).

5.2.3 Raffinamento con 2-opt

Il percorso migliore trovato dal Nearest Neighbor viene poi raffinato con l'euristica di ricerca locale **2-opt** (`twoOpt(path)`), una tecnica standard per il TSP:

- si considerano tutte le coppie di posizioni (`i`, `j`) nel percorso;

- per ciascuna coppia si costruisce un nuovo percorso **invertendo il segmento** tra **i** e **j** (questo "disfa" due archi del percorso e ne crea due nuovi, eliminando eventuali incroci);
- se il nuovo percorso ha un costo totale inferiore, lo si adotta come nuovo "migliore" e si segna che è avvenuto un miglioramento;
- il processo si ripete (ciclo **while (improved)**) finché in una intera scansione non si trova più alcun miglioramento (convergenza a un **ottimo locale 2-opt**).

Questo è l'algoritmo classico di miglioramento locale usato in praticamente tutti i risolutori euristici di TSP: elimina in modo iterativo gli "incroci" nel percorso, tipicamente riducendo il costo del 5-15% rispetto al solo Nearest Neighbor. La complessità di una singola scansione è $O(n^2)$ (coppie **i, j**) moltiplicata per il costo di valutare il percorso ($O(n)$ in **cost()**), quindi $O(n^3)$ per scansione, ripetuta finché non converge — in pratica poche iterazioni per istanze di dimensioni moderate.

5.3 Perché due strategie diverse

Tappe	Algoritmo	Garanzia	Complessità
≤ 12	Held-Karp (DP esatta)	Ottimo globale garantito	Esponenziale ma gestibile ($O(2^n n^2)$)
> 12	Nearest Neighbor multi-start + 2-opt	Nessuna garanzia di ottimo, ma buona qualità pratica	Polinomiale ($O(n^3)$)

La soglia di 12 tappe è il classico compromesso: oltre questo numero, 2^n cresce troppo rapidamente ($2^{13} = 8192$, $2^{20} \approx 1.000.000...$) per essere calcolato in tempo utile nel thread principale del browser (JavaScript è single-threaded e bloccherebbe l'interfaccia), mentre l'euristica scala molto meglio pur restituendo tipicamente soluzioni entro pochi punti percentuali dall'ottimo.

6. Visualizzazione: mappa e profilo altimetrico

- **Mappa:** Leaflet con tile OpenStreetMap standard. Il percorso ottimale viene disegnato come layer GeoJSON (`L.geoJSON`) colorato di verde; i marker vengono ricreati (`rebuildMarkers`) nell'ordine ottimale trovato, con icone numerate e simboli speciali (🚩 partenza, 🏁 arrivo).
- **Profilo altimetrico:** disegnato manualmente su un elemento `<canvas>` (nessuna libreria di grafici). Il disegno tiene conto del **device pixel ratio** per la nitidezza su schermi retina, ridisegna una griglia orizzontale di riferimento, un'area riempita sotto la curva e la linea del profilo, e legge dinamicamente i colori dalle **variabili CSS** del tema (chiaro/scuro) tramite `getComputedStyle`, così da restare coerente con il tema attivo senza duplicare la logica dei colori in JavaScript.
- Il pannello altimetrico è **ridimensionabile** trascinando una maniglia (drag verticale, gestito con `mousedown/touchstart` + listener su `mousemove/touchmove`), e **richiudibile** con doppio click sul titolo.
- Un `ResizeObserver` sul contenitore della mappa e uno sul contenitore del profilo assicurano che, ad ogni ridimensionamento (inclusa l'apertura/chiusura della sidebar), Leaflet richiami `invalidateSize()` e il grafico venga ridisegnato alle nuove dimensioni.

7. Osservazioni, limiti e possibili miglioramenti

- **Doppia chiamata a `/route`:** come notato al §4.5, `draw()` e `fetchElevationProfile()` interrogano entrambi OSRM Route con parametri identici; si potrebbe passare la geometria già ottenuta in `draw()` direttamente a `fetchElevationProfile()`, dimezzando le chiamate di rete per ogni calcolo.
- **Etichetta "Concorde":** fuorviante, dato che l'algoritmo esatto implementato è Held-Karp e non il risolutore Concorde TSP (un software C completamente diverso, non presente nel codice).
- **Approssimazione delle distanze locali:** il fattore fisso `111.32` km/grado usato nel calcolo delle distanze per il profilo altimetrico è corretto solo per la latitudine; introduce un piccolo errore sulla componente longitudinale che cresce allontanandosi dall'equatore (trascurabile alle latitudini italiane, ma concettualmente impreciso).
- **Dipendenza da servizi pubblici gratuiti:** sia il server demo di OSRM (`router.project-osrm.org`) sia Open-Elevation sono istanze pubbliche con limiti di traffico non garantiti; per un uso in produzione andrebbero sostituiti con istanze proprie o piani a pagamento.
- **2-opt senza limite di tempo:** per un numero di tappe grande (es. centinaia), il ciclo `while(improved)` di `twoOpt` potrebbe richiedere molte iterazioni; non è presente un limite massimo di iterazioni o un timeout, il che in casi estremi potrebbe rallentare l'interfaccia (single-thread).
- **Nessuna persistenza:** le tappe inserite non vengono salvate (né in `localStorage` né altrove); ricaricando la pagina lo stato si perde.

8. Conclusione

OpenTravel è un'applicazione didatticamente interessante perché mostra, in poche centinaia di righe di JavaScript "vanilla", una pipeline completa che combina:

- geocoding di indirizzi in linguaggio naturale,
- richiesta di una matrice di costi reali stradali a un motore di routing,
- risoluzione di un problema di ottimizzazione combinatoria classico (TSP) con due strategie complementari — esatta per istanze piccole, euristica per istanze grandi,
- tracciamento del percorso reale su rete stradale e arricchimento con dati di elevazione,
- visualizzazione interattiva su mappa e grafico disegnato a mano su canvas.

L'architettura, pur semplice (nessun framework, nessun backend proprio), è ben organizzata attorno a un'unica classe di stato e sfrutta in modo efficace API pubbliche dell'ecosistema OpenStreetMap.